

Head Pose (Yaw) Regressor Model Evaluation

This notebook compares different CNN architectures for head yaw angle regression:

1. Attention-based CNN with CBAM (Convolutional Block Attention Module)
2. Custom ResNet-based CNN
3. Transfer Learning with pretrained ResNet50
4. Transfer Learning with pretrained MobileNetV2

```
In [1]: import keras
keras.config.enable_unsafe_deserialization()

import numpy as np
import pandas as pd
import tensorflow as tf
import matplotlib.pyplot as plt
import os
from tensorflow import keras
from tensorflow.keras import backend as K
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential, Model, load_model
from tensorflow.keras.layers import Add, Lambda, GlobalAveragePooling2D, GlobalMaxPooling2D, Input, Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization, Activation, Add, GlobalAveragePooling2D, Multiply, Reshape, Permute, Concatenate
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import MeanSquaredError
from tensorflow.keras.callbacks import EarlyStopping, ReduceLRonPlateau, ModelCheckpoint
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.regularizers import l2
from tensorflow.keras.applications import ResNet50, MobileNetV2

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import KFold
```

Load the preprocessed dataset

```
In [2]: # Load the preprocessed dataset
with open(' ../datasets/regression/data_20000.npy', 'rb') as data, open(' ../datasets/regression/label_20000.npy', 'rb') as label:
    dataset = np.load(data)
    print(dataset.shape)
    labels = np.load(label)
    print(labels.shape)
```

(20000, 128, 128, 3)
(20000, 3)

Join both the dataset and labels into a single dataframe

This dataset contains images and their yaw values

```
In [3]: # Extract yaw values only
yaw_labels = labels[:, 1]
df = pd.DataFrame({
    'image': list(dataset),
    'label': yaw_labels
})

df.head()
```

Out [3]:

	image	label
0	[[[0. 0. 0.] [0. 0. 0.] [0. 0. 0.] [0. 0. 0.] ... 30.915585	
1	[[[0. 0. 0.] [0. 0. 0.] [0. 0. 0.] [0. 0. 0.] ... -70.093781	
2	[[[86. 96. 98.] [86. 96. 98.] [86. 96. 97.] [8... -59.824131	
3	[[[0. 0. 0.] [0. 0. 0.] [0. 0. 0.] [0. 0. 0.] ... 77.033363	
4	[[[0. 0. 0.] [0. 0. 0.] [0. 0. 0.] [29. 29. 29... -60.158260	

Split the dataset into training, testing and validation

```
In [4]: validation_split = 0.4

num_train_samples = int((1 - validation_split) * len(df))
testing = df[num_train_samples:]
training = df[:num_train_samples]

num_validation_samples = int(len(testing) * 0.5)
testing = testing[num_validation_samples:]
validation = testing[num_validation_samples:]

print(f'Training: {training.shape[0]}')
print(f'Testing: {testing.shape[0]}')
print(f'Validation: {validation.shape[0]}')

Training: 12000
Testing: 4000
Validation: 4000
```

Define training, testing and validation data

```
In [5]: # x values
x_training = training['image']
x_testing = testing['image']
x_validation = validation['image']

# y values
y_training = training['label']
y_testing = testing['label']
y_validation = validation['label']

x_training = np.array([np.array(xi).astype(np.float32) for xi in x_training])
y_training = np.array([np.array(yi).astype(np.float32) for yi in y_training])
x_validation = np.array([np.array(xi).astype(np.float32) for xi in x_validation])
y_validation = np.array([np.array(yi).astype(np.float32) for yi in y_validation])
x_testing = np.array([np.array(xi).astype(np.float32) for xi in x_testing])
y_testing = np.array([np.array(yi).astype(np.float32) for yi in y_testing])

# Normalize the data to [0, 1] range
x_training = x_training.astype('float32') / 255.0
x_validation = x_validation.astype('float32') / 255.0
x_testing = x_testing.astype('float32') / 255.0
```

Define training callbacks and data augmentation

```
In [6]: # Function to ensure directory exists
def ensure_dir_exists(directory):
    if not os.path.exists(directory):
        os.makedirs(directory)
        print(f"Created directory: {directory}")

# Callbacks for training
def get_callbacks(model_name):
    early_stopping = EarlyStopping(
        monitor='val_loss',
        patience=8,
        min_delta=0.001,
        mode='min',
        restore_best_weights=True
    )

    reduce_lr = ReduceLRonPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=4,
        min_lr=0.00001
    )

    # Make sure experiments directory exists
    output_dir = '../trained_models'
    ensure_dir_exists(output_dir)

    model_checkpoint = ModelCheckpoint(
        f'{output_dir}/{model_name}.keras',
        monitor='val_loss',
        save_best_only=True,
        mode='min'
    )

    return [early_stopping, reduce_lr, model_checkpoint]

# Data augmentation
datagen = ImageDataGenerator(
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True,
    fill_mode='nearest'
)

datagen.fit(x_training)
```

Utility functions for evaluation

```
In [7]: # Function to evaluate models
def evaluate_model(model, x_test, y_test, model_name):
    predictions = model.predict(x_test)

    mae = mean_absolute_error(y_test, predictions)
    mse = mean_squared_error(y_test, predictions)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, predictions)

    print(f'==== {model_name} Evaluation =====')
    print(f'MAE: {mae}')
    print(f'MSE: {mse}')
    print(f'RMSE: {rmse}')
    print(f'R-Squared: {r2}')

# Plot predicted vs actual
plt.figure(figsize=(10, 6))
plt.scatter(y_test, predictions)
plt.xlabel('Actual values')
plt.ylabel('Predicted values')
plt.title(f'{model_name} - Predicted vs. Actual values')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red')
plt.grid(True, alpha=0.3)
plt.show()

# Return metrics as dictionary
return {'model': model_name, 'mae': mae, 'mse': mse, 'rmse': rmse, 'r2': r2}

# Function to plot training history
def plot_training_history(history, model_name):
    metrics = pd.DataFrame(history.history)

    # Plot loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(metrics['loss'], label='Training Loss')
    plt.plot(metrics['val_loss'], label='Validation Loss')
    plt.title(f'{model_name} - Training vs Validation Loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.grid(True, alpha=0.3)

    # Plot MAE
    plt.subplot(1, 2, 2)
    plt.plot(metrics['mae'], label='Training MAE')
    plt.plot(metrics['val_mae'], label='Validation MAE')
    plt.title(f'{model_name} - Training vs Validation MAE')
    plt.xlabel('Epochs')
    plt.ylabel('MAE')
    plt.legend()
    plt.grid(True, alpha=0.3)

    plt.tight_layout()

    # Save the training history plot
    history_dir = '../model/experiments'
    ensure_dir_exists(history_dir)
    plt.savefig(f'{history_dir}/{model_name}_training_history.png')

    plt.show()

    # Save training history as JSON for future reference
    history_data = {k: [float(val) for val in v] for k, v in history.history.items()}
    import json
    with open(f'{history_dir}/{model_name}_training_history.json', 'w') as f:
        json.dump(history_data, f)

    return metrics
```

1. Attention-based CNN with CBAM

This implementation uses a Convolutional Block Attention Module (CBAM) that helps the model focus on the most relevant parts of the face image. The module consists of:

1. **Channel Attention:** Focuses on "what" is important by weighting each channel
2. **Spatial Attention:** Focuses on "where" is important by weighting each spatial location

The key improvement here is to use named functions instead of Lambda layers for better compatibility.

```
In [8]: # Define named functions for the operations instead of lambda functions
def mean_channel(x, axis=-1, keepdims=True):
    return K.mean(x, axis=axis, keepdims=keepdims)

def max_channel(x, axis=-1, keepdims=True):
    return K.max(x, axis=axis, keepdims=keepdims)

# Register the custom functions
tf.keras.utils.get_custom_objects().update({'mean_channel': mean_channel})
tf.keras.utils.get_custom_objects().update({'max_channel': max_channel})

In [9]: def channel_attention(x, ratio=8):
    channel_axis = -1
    filters = x.shape[channel_axis]
    if filters is None or filters < ratio :
        mlp_filters_reduced = max(1, int(filters / 2)) if filters else 1
    else:
        mlp_filters_reduced = filters // ratio

    # Shared MLP - defining layers once for potential true sharing if desired later,
    # or use distinct names if not strictly sharing weights yet.
    # For this example, we'll keep them as distinct instances within each path for simplicity of the current fix.
    mlp_reduced_avg = Conv2D(mlp_filters_reduced, kernel_size=1, use_bias=True, activation='relu', name=f'{x.name.split(":")[0]}_channel_mlp_reduced_avg')
    mlp_expanded_avg = Conv2D(filters, kernel_size=1, use_bias=True, name=f'{x.name.split(":")[0]}_channel_mlp_expanded_avg')

    mlp_reduced_max = Conv2D(mlp_filters_reduced, kernel_size=1, use_bias=True, activation='relu', name=f'{x.name.split(":")[0]}_channel_mlp_reduced_max')
    mlp_expanded_max = Conv2D(filters, kernel_size=1, use_bias=True, name=f'{x.name.split(":")[0]}_channel_mlp_expanded_max')

    # Average Pooling Path
    avg_pool = GlobalAveragePooling2D(name=f'{x.name.split(":")[0]}_channel_avg_pool')(x)
    avg_pool = Reshape((1, 1, filters), name=f'{x.name.split(":")[0]}_channel_reshape_avg')(avg_pool)
    avg_pool = mlp_reduced_avg(avg_pool)
    avg_pool = mlp_expanded_avg(avg_pool)

    # Max Pooling Path
    max_pool = GlobalMaxPooling2D(name=f'{x.name.split(":")[0]}_channel_max_pool')(x)
    max_pool = Reshape((1, 1, filters), name=f'{x.name.split(":")[0]}_channel_reshape_max')(max_pool)
    max_pool = mlp_reduced_max(max_pool)
    max_pool = mlp_expanded_max(max_pool)

    summed_features = Add(name=f'{x.name.split(":")[0]}_channel_add')((avg_pool, max_pool))
    channel_weights = Activation('sigmoid', name=f'{x.name.split(":")[0]}_channel_sigmoid')(summed_features)

    return Multiply(name=f'{x.name.split(":")[0]}_channel_multiply')((x, channel_weights))

def spatial_attention(x, kernel_size=7):
    channel_axis = -1 # 'channels_last'

    # Using named functions instead of Lambda layers
    avg_pool = Lambda(
        mean_channel,
        output_shape=lambda s: (s[0], s[1], s[2], 1),
        arguments={'axis': channel_axis, 'keepdims': True},
        name=f'{x.name.split(":")[0]}_spatial_avg_pool_lambda'
    )(x)

    max_pool = Lambda(
        max_channel,
        output_shape=lambda s: (s[0], s[1], s[2], 1),
        arguments={'axis': channel_axis, 'keepdims': True},
        name=f'{x.name.split(":")[0]}_spatial_max_pool_lambda'
    )(x)

    concat = Concatenate(axis=channel_axis,
        name=f'{x.name.split(":")[0]}_spatial_concat')((avg_pool, max_pool))

    attention_feature_map = Conv2D(
        1, kernel_size=kernel_size, strides=1, padding='same',
        activation='sigmoid', use_bias=False,
        name=f'{x.name.split(":")[0]}_spatial_conv_sigmoid'
    )(concat)

    return Multiply(name=f'{x.name.split(":")[0]}_spatial_multiply')((x, attention_feature_map))

def cbam_block(x, ratio=8, kernel_size=7):
    x_channel = channel_attention(x, ratio)
    x_spatial = spatial_attention(x_channel, kernel_size)
    return x_spatial
```

```
In [10]: def create_attention_cnn_model(input_shape=(128, 128, 3)):
    inputs = Input(shape=input_shape)

    # First convolutional block
    x = Conv2D(32, (3, 3), activation='relu', padding='same')(inputs)
    x = BatchNormalization()(x)
    x = Conv2D(32, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D(pool_size=(2, 2))(x)

    # Apply attention block
    x = cbam_block(x)

    # Second convolutional block
    x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = Conv2D(64, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D(pool_size=(2, 2))(x)

    # Apply attention block
    x = cbam_block(x)

    # Third convolutional block
    x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = Conv2D(128, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D(pool_size=(2, 2))(x)

    # Apply attention block
    x = cbam_block(x)

    # Fourth convolutional block
    x = Conv2D(256, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = Conv2D(256, (3, 3), activation='relu', padding='same')(x)
    x = BatchNormalization()(x)
    x = MaxPooling2D(pool_size=(2, 2))(x)

    # Apply attention block
    x = cbam_block(x)

    # Fully connected layers
    x = Flatten()(x)
    x = Dense(512, activation='relu')(x)
    x = Dropout(0.5)(x)
    x = Dense(128, activation='relu')(x)
    x = Dropout(0.3)(x)
    outputs = Dense(1)(x) # Regression output for yaw angle

    model = Model(inputs=inputs, outputs=outputs)

    # Compile the model
    model.compile(
        optimizer=Adam(learning_rate=0.001),
        loss='mse',
        metrics=['mae']
    )

    return model
```



```
In [11]: # Create attention-based CNN model
attention_model = create_attention_cnn_model()
attention_model.summary()

2025-06-07 15:21:39.886978: I metal_plugin/src/device/metal_device.cc:1154] Metal device set to: Apple M1 Pro
2025-06-07 15:21:39.887853: I metal_plugin/src/device/metal_device.cc:296] systemMemory: 16.00 GB
2025-06-07 15:21:39.887879: I metal_plugin/src/device/metal_device.cc:313] maxCacheSize: 5.33 GB
2025-06-07 15:21:39.887343: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:385] Could not identify NUMA node of platform GPU ID 0, defaulting to 0. Your kernel may not have been built with NUMA support.
2025-06-07 15:21:39.887398: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271] Created TensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 0 MB memory) -> physical PluggableDevice (device: 0, name: METAL, pci bus id: <undefined>)

Model: "functional"
```

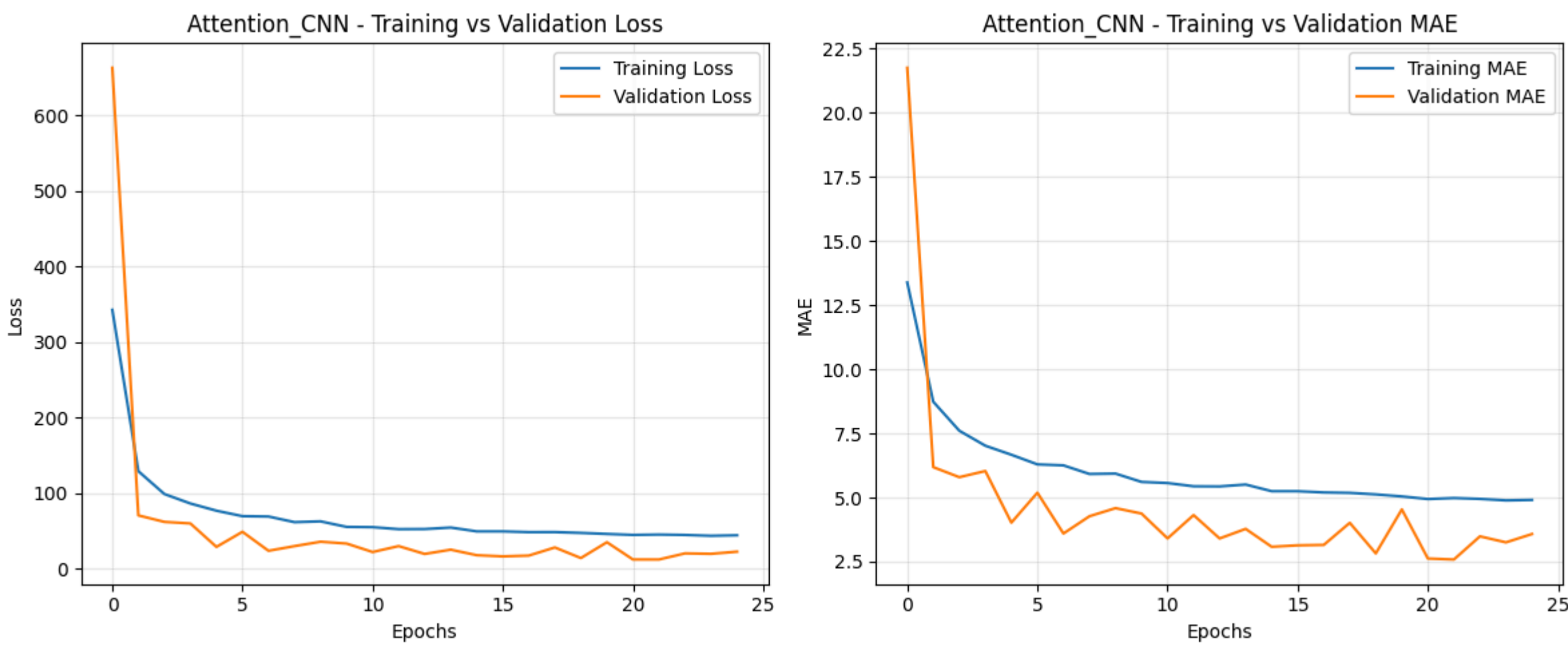

Layer (type)	Output Shape	Param #	Connected to
Input_layer (InputLayer)	(None, 128, 128, 3)	0	-
conv2d (Conv2D)	(None, 128, 128, 32)	896	input_layer[0][0]
batch_normalization (BatchNormalization)	(None, 128, 128, 32)	128	conv2d[0][0]
conv2d_1 (Conv2D)	(None, 128, 128, 32)	9,248	batch_normalizat...
batch_normalization (BatchNormalization)	(None, 128, 128, 32)	128	conv2d_1[0][0]
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0	batch_normalizat...
keras_tensor_5_cha... (GlobalAveragePool...	(None, 32)	0	max_pooling2d[0]...
keras_tensor_5_cha... (GlobalMaxPooling2...	(None, 32)	0	max_pooling2d[0]...
keras_tensor_5_cha... (Reshape)	(None, 1, 1, 32)	0	keras_tensor_5_c...
keras_tensor_5_cha... (Reshape)	(None, 1, 1, 32)	0	keras_tensor_5_c...
keras_tensor_5_cha... (Conv2D)	(None, 1, 1, 4)	132	keras_tensor_5_c...
keras_tensor_5_cha... (Conv2D)	(None, 1, 1, 4)	132	keras_tensor_5_c...
keras_tensor_5_cha... (Conv2D)	(None, 1, 1, 32)	160	keras_tensor_5_c...
keras_tensor_5_cha... (Conv2D)	(None, 1, 1, 32)	160	keras_tensor_5_c...
keras_tensor_5_cha... (Add)	(None, 1, 1, 32)	0	keras_tensor_5_c... keras_tensor_5_c...
keras_tensor_5_cha... (Activation)	(None, 1, 1, 32)	0	keras_tensor_5_c...
keras_tensor_5_cha... (Multiply)	(None, 64, 64, 32)	0	max_pooling2d[0]... keras_tensor_5_c...
keras_tensor_16_sp... (Lambda)	(None, 64, 64, 1)	0	keras_tensor_5_c...
keras_tensor_16_sp... (Lambda)	(None, 64, 64, 1)	0	keras_tensor_5_c...
keras_tensor_16_sp... (Concatenate)	(None, 64, 64, 2)	0	keras_tensor_16_... keras_tensor_16_...
keras_tensor_16_sp... (Conv2D)	(None, 64, 64, 1)	98	keras_tensor_16_...
keras_tensor_16_sp... (Multiply)	(None, 64, 64, 32)	0	keras_tensor_5_c... keras_tensor_16_...
conv2d_2 (Conv2D)	(None, 64, 64, 64)	18,496	keras_tensor_16_...
batch_normalization (BatchNormalization)	(None, 64, 64, 64)	256	conv2d_2[0][0]
conv2d_3 (Conv2D)	(None, 64, 64, 64)	36,928	batch_normalizat...
batch_normalization (BatchNormalization)	(None, 64, 64, 64)	256	conv2d_3[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0	batch_normalizat...
keras_tensor_26_ch... (GlobalAveragePool...	(None, 64)	0	max_pooling2d_1[...
keras_tensor_26_ch... (GlobalMaxPooling2...	(None, 64)	0	max_pooling2d_1[...
keras_tensor_26_ch... (Reshape)	(None, 1, 1, 64)	0	keras_tensor_26_...
keras_tensor_26_ch... (Reshape)	(None, 1, 1, 64)	0	keras_tensor_26_...
keras_tensor_26_ch... (Conv2D)	(None, 1, 1, 8)	520	keras_tensor_26_...
keras_tensor_26_ch... (Conv2D)	(None, 1, 1, 8)	520	keras_tensor_26_...
keras_tensor_26_ch... (Conv2D)	(None, 1, 1, 64)	576	keras_tensor_26_...
keras_tensor_26_ch... (Conv2D)	(None, 1, 1, 64)	576	keras_tensor_26_...
keras_tensor_26_ch... (Add)	(None, 1, 1, 64)	0	keras_tensor_26_... keras_tensor_26_...
keras_tensor_26_ch... (Activation)	(None, 1, 1, 64)	0	keras_tensor_26_...
keras_tensor_26_ch... (Multiply)	(None, 32, 32, 64)	0	max_pooling2d_1[... keras_tensor_26_...
keras_tensor_37_sp... (Lambda)	(None, 32, 32, 1)	0	keras_tensor_26_...
keras_tensor_37_sp... (Lambda)	(None, 32, 32, 1)	0	keras_tensor_26_...
keras_tensor_37_sp... (Concatenate)	(None, 32, 32, 2)	0	keras_tensor_37_... keras_tensor_37_...
keras_tensor_37_sp... (Conv2D)	(None, 32, 32, 1)	98	keras_tensor_37_...
keras_tensor_37_sp... (Multiply)	(None, 32, 32, 64)	0	keras_tensor_26_... keras_tensor_37_...
conv2d_4 (Conv2D)	(None, 32, 32, 128)	73,856	keras_tensor_37_...
batch_normalization (BatchNormalization)	(None, 32, 32, 128)	512	conv2d_4[0][0]
conv2d_5 (Conv2D)	(None, 32, 32, 128)	147,584	batch_normalizat...
batch_normalization (BatchNormalization)	(None, 32, 32, 128)	512	conv2d_5[0][0]
max_pooling2d_2 (MaxPooling2D)	(None, 16, 16, 128)	0	batch_normalizat...
keras_tensor_47_ch... (GlobalAveragePool...	(None, 128)	0	max_pooling2d_2[...
keras_tensor_47_ch... (GlobalMaxPooling2...	(None, 128)	0	max_pooling2d_2[...
keras_tensor_47_ch... (Reshape)	(None, 1, 1, 128)	0	keras_tensor_47_...
keras_tensor_47_ch... (Reshape)	(None, 1, 1, 128)	0	keras_tensor_47_...
keras_tensor_47_ch... (Conv2D)	(None, 1, 1, 16)	2,064	keras_tensor_47_...
keras_tensor_47_ch... (Conv2D)	(None, 1, 1, 16)	2,064	keras_tensor_47_...
keras_tensor_47_ch... (Conv2D)	(None, 1, 1, 128)	2,176	keras_tensor_47_...
keras_tensor_47_ch... (Conv2D)	(None, 1, 1, 128)	2,176	keras_tensor_47_...
keras_tensor_47_ch... (Add)	(None, 1, 1, 128)	0	keras_tensor_47_... keras_tensor_47_...
keras_tensor_47_ch... (Activation)	(None, 1, 1, 128)	0	keras_tensor_47_...
keras_tensor_47_ch... (Multiply)	(None, 16, 16, 128)	0	max_pooling2d_2[... keras_tensor_47_...
keras_tensor_58_sp... (Lambda)	(None, 16, 16, 1)	0	keras_tensor_47_...
keras_tensor_58_sp... (Lambda)	(None, 16, 16, 1)	0	keras_tensor_47_...
keras_tensor_58_sp... (Concatenate)	(None, 16, 16, 2)	0	keras_tensor_58_... keras_tensor_58_...
keras_tensor_58_sp... (Conv2D)	(None, 16, 16, 1)	98	keras_tensor_58_...
keras_tensor_58_sp... (Multiply)	(None, 16, 16, 128)	0	keras_tensor_47_... keras_tensor_58_...
conv2d_6 (Conv2D)	(None, 16, 16, 256)	295,168	keras_tensor_58_...
batch_normalization (BatchNormalization)	(None, 16, 16, 256)	1,024	conv2d_6[0][0]
conv2d_7 (Conv2D)	(None, 16, 16, 256)	590,080	batch_normalizat...
batch_normalization (BatchNormalization)	(None, 16, 16, 256)	1,024	conv2d_7[0][0]
max_pooling2d_3 (MaxPooling2D)	(None, 8, 8, 256)	0	batch_normalizat...
keras_tensor_68_ch... (GlobalAveragePool...	(None, 256)	0	max_pooling2d_3[...
keras_tensor_68_ch... (GlobalMaxPooling2...	(None, 256)	0	max_pooling2d_3[...
keras_tensor_68_ch... (Reshape)	(None, 1, 1, 256)	0	keras_tensor_68_...
keras_tensor_68_ch... (Reshape)	(None, 1, 1, 256)	0	keras_tensor_68_...
keras_tensor_68_ch... (Conv2D)	(None, 1, 1, 32)	8,224	keras_tensor_68_...
keras_tensor_68_ch... (Conv2D)	(None, 1, 1, 32)	8,224	keras_tensor_68_...
keras_tensor_68_ch... (Conv2D)	(None, 1, 1, 256)	8,448	keras_tensor_68_...
keras_tensor_68_ch... (Conv2D)	(None, 1, 1, 256)	8,448	keras_tensor_68_...
keras_tensor_68_ch... (Add)	(None, 1, 1, 256)	0	keras_tensor_68_... keras_tensor_68_...
keras_tensor_68_ch... (Activation)	(None, 1, 1, 256)	0	keras_tensor_68_...
keras_tensor_68_ch... (Multiply)	(None, 8, 8, 256)	0	max_pooling2d_3[... keras_tensor_68_...
keras_tensor_79_sp... (Lambda)	(None, 8, 8, 1)	0	keras_tensor_68_...
keras_tensor_79_sp... (Lambda)	(None, 8, 8, 1)	0	keras_tensor_68_...
keras_tensor_79_sp... (Concatenate)	(None, 8, 8, 2)	0	keras_tensor_79_... keras_tensor_79_...
keras_tensor_79_sp... (Conv2D)	(None, 8, 8, 1)	98	keras_tensor_79_...
keras_tensor_79_sp... (Multiply)	(None, 8, 8, 256)	0	keras_tensor_68_... keras_tensor_79_...
flatten (Flatten)	(None, 16384)	0	keras_tensor_79_...
dense (Dense)	(None, 512)	8,389,120	flatten[0][0]
dropout (Dropout)	(None, 512)	0	dense[0][0]
dense_1 (Dense)	(None, 128)	65,664	dropout[0][0]
dropout_1 (Dropout)	(None, 128)	0	dense_1[0][0]
dense_2 (Dense)	(None, 1)	129	dropout_1[0][0]
Total params: 9,676,001 (36.91 MB)			
Trainable params: 9,674,081 (36.90 MB)			

Non-trainable params: 1,920 (7.50 KB)

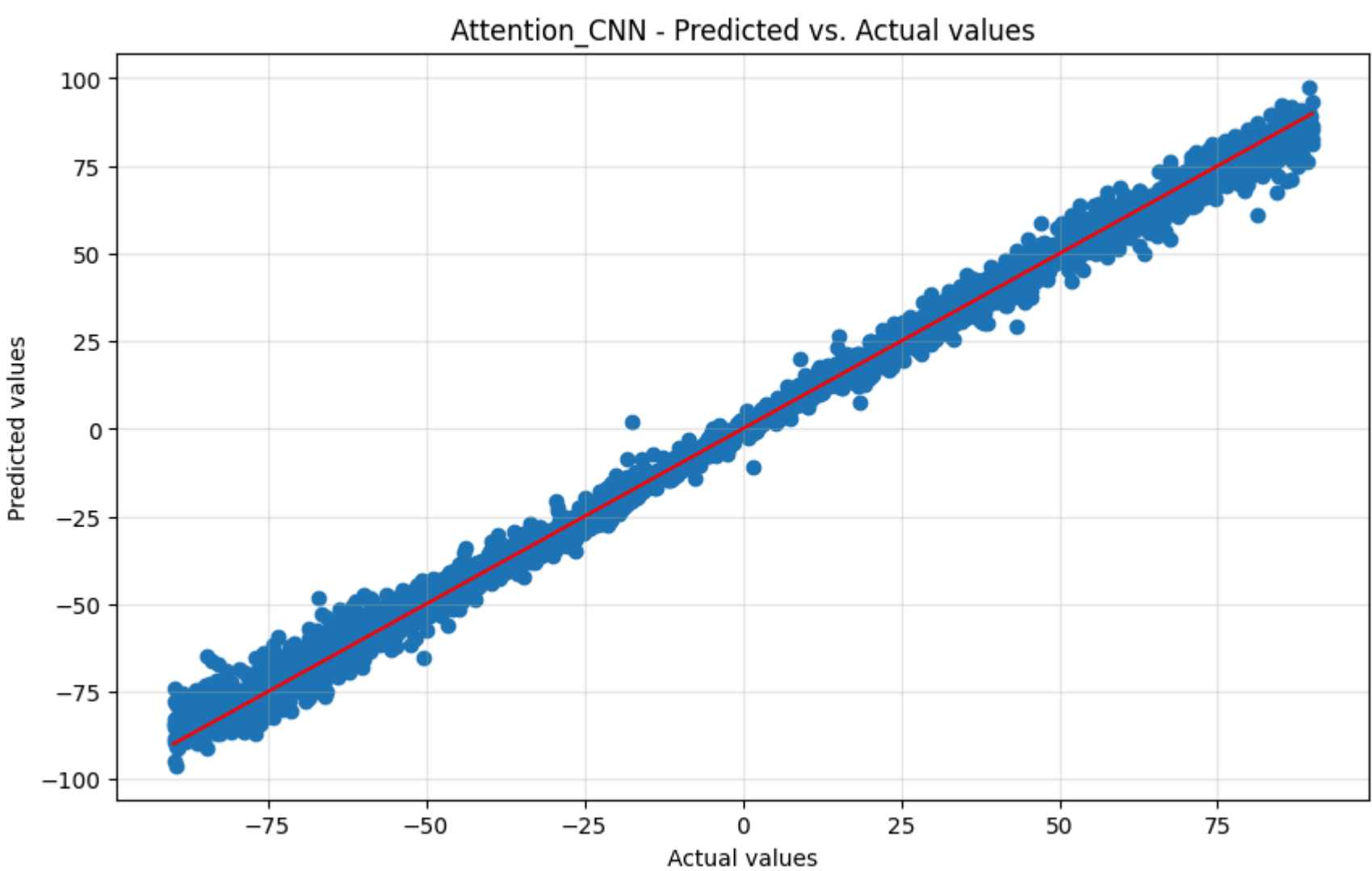
```
In [12]: # Train attention-based CNN model
attention_history = attention_model.fit(
    x_training,
    y_training,
    epochs=25,
    batch_size=32,
    validation_data=(x_validation, y_validation),
    callbacks=get_callbacks('attention_cnn_yaw_regressor')
)

Epoch 1/25
2025-06-07 15:23:13.680699: I tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117] Plugin optimizer for device_type GPU is enabled.
375/375 ----- 98s 237ms/step - loss: 689.7928 - mae: 18.7842 - val_loss: 663.0005 - val_mae: 21.7484 - learning_rate: 0.0010
Epoch 2/25
375/375 ----- 84s 224ms/step - loss: 135.3927 - mae: 8.9887 - val_loss: 70.6734 - val_mae: 6.1844 - learning_rate: 0.0010
Epoch 3/25
375/375 ----- 85s 225ms/step - loss: 102.1918 - mae: 7.7284 - val_loss: 62.0464 - val_mae: 5.7933 - learning_rate: 0.0010
Epoch 4/25
375/375 ----- 84s 223ms/step - loss: 88.0437 - mae: 7.1152 - val_loss: 59.9913 - val_mae: 6.0333 - learning_rate: 0.0010
Epoch 5/25
375/375 ----- 84s 223ms/step - loss: 78.7190 - mae: 6.8048 - val_loss: 28.9502 - val_mae: 4.0215 - learning_rate: 0.0010
Epoch 6/25
375/375 ----- 82s 218ms/step - loss: 70.6094 - mae: 6.3752 - val_loss: 48.9226 - val_mae: 5.1890 - learning_rate: 0.0010
Epoch 7/25
375/375 ----- 84s 223ms/step - loss: 72.2387 - mae: 6.3686 - val_loss: 23.7881 - val_mae: 3.5976 - learning_rate: 0.0010
Epoch 8/25
375/375 ----- 83s 221ms/step - loss: 62.6086 - mae: 5.9770 - val_loss: 29.9804 - val_mae: 4.2709 - learning_rate: 0.0010
Epoch 9/25
375/375 ----- 83s 220ms/step - loss: 64.1416 - mae: 5.9888 - val_loss: 35.7907 - val_mae: 4.5863 - learning_rate: 0.0010
Epoch 10/25
375/375 ----- 85s 226ms/step - loss: 55.5365 - mae: 5.6322 - val_loss: 33.3818 - val_mae: 4.3760 - learning_rate: 0.0010
Epoch 11/25
375/375 ----- 87s 232ms/step - loss: 54.2607 - mae: 5.5259 - val_loss: 22.1576 - val_mae: 3.4073 - learning_rate: 0.0010
Epoch 12/25
375/375 ----- 86s 227ms/step - loss: 54.0297 - mae: 5.5363 - val_loss: 30.0056 - val_mae: 4.3150 - learning_rate: 0.0010
Epoch 13/25
375/375 ----- 85s 226ms/step - loss: 52.7361 - mae: 5.4413 - val_loss: 19.6204 - val_mae: 3.4047 - learning_rate: 0.0010
Epoch 14/25
375/375 ----- 84s 224ms/step - loss: 55.4624 - mae: 5.5583 - val_loss: 25.2666 - val_mae: 3.7787 - learning_rate: 0.0010
Epoch 15/25
375/375 ----- 87s 233ms/step - loss: 49.5819 - mae: 5.2361 - val_loss: 17.9571 - val_mae: 3.0805 - learning_rate: 0.0010
Epoch 16/25
375/375 ----- 89s 237ms/step - loss: 50.0471 - mae: 5.2881 - val_loss: 16.3878 - val_mae: 3.1369 - learning_rate: 0.0010
Epoch 17/25
375/375 ----- 84s 223ms/step - loss: 48.7730 - mae: 5.2170 - val_loss: 17.4568 - val_mae: 3.1531 - learning_rate: 0.0010
Epoch 18/25
375/375 ----- 84s 223ms/step - loss: 48.7228 - mae: 5.1979 - val_loss: 28.2057 - val_mae: 4.0142 - learning_rate: 0.0010
Epoch 19/25
375/375 ----- 86s 230ms/step - loss: 47.2125 - mae: 5.1195 - val_loss: 14.1799 - val_mae: 2.8224 - learning_rate: 0.0010
Epoch 20/25
375/375 ----- 86s 230ms/step - loss: 47.2799 - mae: 5.1035 - val_loss: 35.1368 - val_mae: 4.5372 - learning_rate: 0.0010
Epoch 21/25
375/375 ----- 84s 223ms/step - loss: 46.5883 - mae: 5.0082 - val_loss: 12.2904 - val_mae: 2.6233 - learning_rate: 0.0010
Epoch 22/25
375/375 ----- 88s 233ms/step - loss: 44.8786 - mae: 4.9697 - val_loss: 12.2704 - val_mae: 2.5851 - learning_rate: 0.0010
Epoch 23/25
375/375 ----- 86s 228ms/step - loss: 44.2658 - mae: 4.9184 - val_loss: 20.3572 - val_mae: 3.4871 - learning_rate: 0.0010
Epoch 24/25
375/375 ----- 87s 232ms/step - loss: 43.9432 - mae: 4.9011 - val_loss: 19.7637 - val_mae: 3.2524 - learning_rate: 0.0010
Epoch 25/25
375/375 ----- 87s 232ms/step - loss: 43.3333 - mae: 4.8576 - val_loss: 22.6142 - val_mae: 3.5786 - learning_rate: 0.0010
```

```
In [13]: # Plot training history and evaluate attention-based CNN model
attention_metrics = plot_training_history(attention_history, 'Attention_CNN')
attention_results = evaluate_model(attention_model, x_testing, y_testing, 'Attention_CNN')
```



125/125 ----- 4s 27ms/step
==== Attention_CNN Evaluation =====
MAE: 2.5850532054901123
MSE: 12.270391464233398
RMSE: 3.502911855847054
R-squared: 0.995864987373352



2. Custom ResNet-based CNN

This model uses a custom architecture with residual connections inspired by ResNet.

```
In [14]: def residual_block(x, filters, kernel_size=3, strides=1, activation='relu'):
# Shortcut connection (identity or projection)
shortcut = x

# First convolution
y = Conv2D(filters, kernel_size=kernel_size, strides=strides, padding='same')(x)
y = BatchNormalization()(y)
y = Activation(activation)(y)

# Second convolution
y = Conv2D(filters, kernel_size=kernel_size, strides=1, padding='same')(y)
y = BatchNormalization()(y)

# If dimensions changed, adjust shortcut with 1x1 conv
if strides != 1 or shortcut.shape[-1] != filters:
    shortcut = Conv2D(filters, kernel_size=1, strides=strides, padding='same')(shortcut)
    shortcut = BatchNormalization()(shortcut)

# Add shortcut to output
y = Add()((shortcut, y))
y = Activation(activation)(y)

return y

In [15]: def create_custom_resnet_model(input_shape=(128, 128, 3)):
inputs = Input(shape=input_shape)

# Initial convolution
x = Conv2D(32, kernel_size=7, strides=2, padding='same')(inputs)
x = BatchNormalization()(x)
x = Activation('relu')(x)
x = MaxPooling2D(pool_size=3, strides=2, padding='same')(x)

# Residual blocks
x = residual_block(x, filters=32)
x = residual_block(x, filters=32)

# Increase filters and downsample
x = residual_block(x, filters=64, strides=2)
x = residual_block(x, filters=64)

# Increase filters and downsample
x = residual_block(x, filters=128, strides=2)
x = residual_block(x, filters=128)

# Increase filters and downsample
x = residual_block(x, filters=256, strides=2)
x = residual_block(x, filters=256)

# Global pooling
x = GlobalAveragePooling2D()(x)

# Fully connected layers
x = Dense(512, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
outputs = Dense(1)(x) # Regression output for yaw angle

model = Model(inputs=inputs, outputs=outputs)

# Compile the model
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mse',
    metrics=['mae']
)

return model
```

```
In [16]: # Create and train custom ResNet model
custom_resnet_model = create_custom_resnet_model()
custom_resnet_model.summary()
```

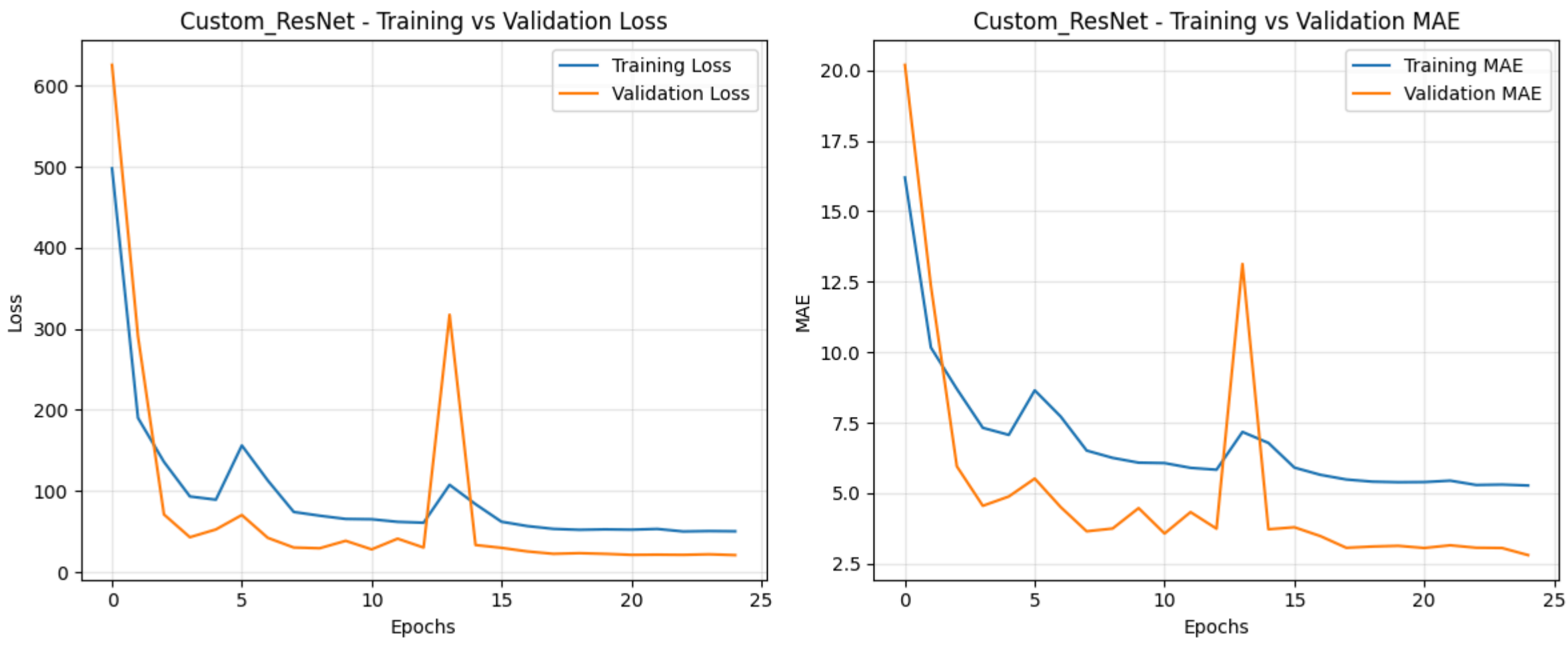
Model: "functional_1"

Layer (type)	Output Shape	Param #	Connected to
input_layer_1 (InputLayer)	(None, 128, 128, 3)	0	-
conv2d_8 (Conv2D)	(None, 64, 64, 32)	4,736	input_layer_1[0]...
batch_normalizatio... (BatchNormalization)	(None, 64, 64, 32)	128	conv2d_8[0][0]
activation (Activation)	(None, 64, 64, 32)	0	batch_normalizat...
max_pooling2d_4 (MaxPooling2D)	(None, 32, 32, 32)	0	activation[0][0]
conv2d_9 (Conv2D)	(None, 32, 32, 32)	9,744	max pooling2d 4[...

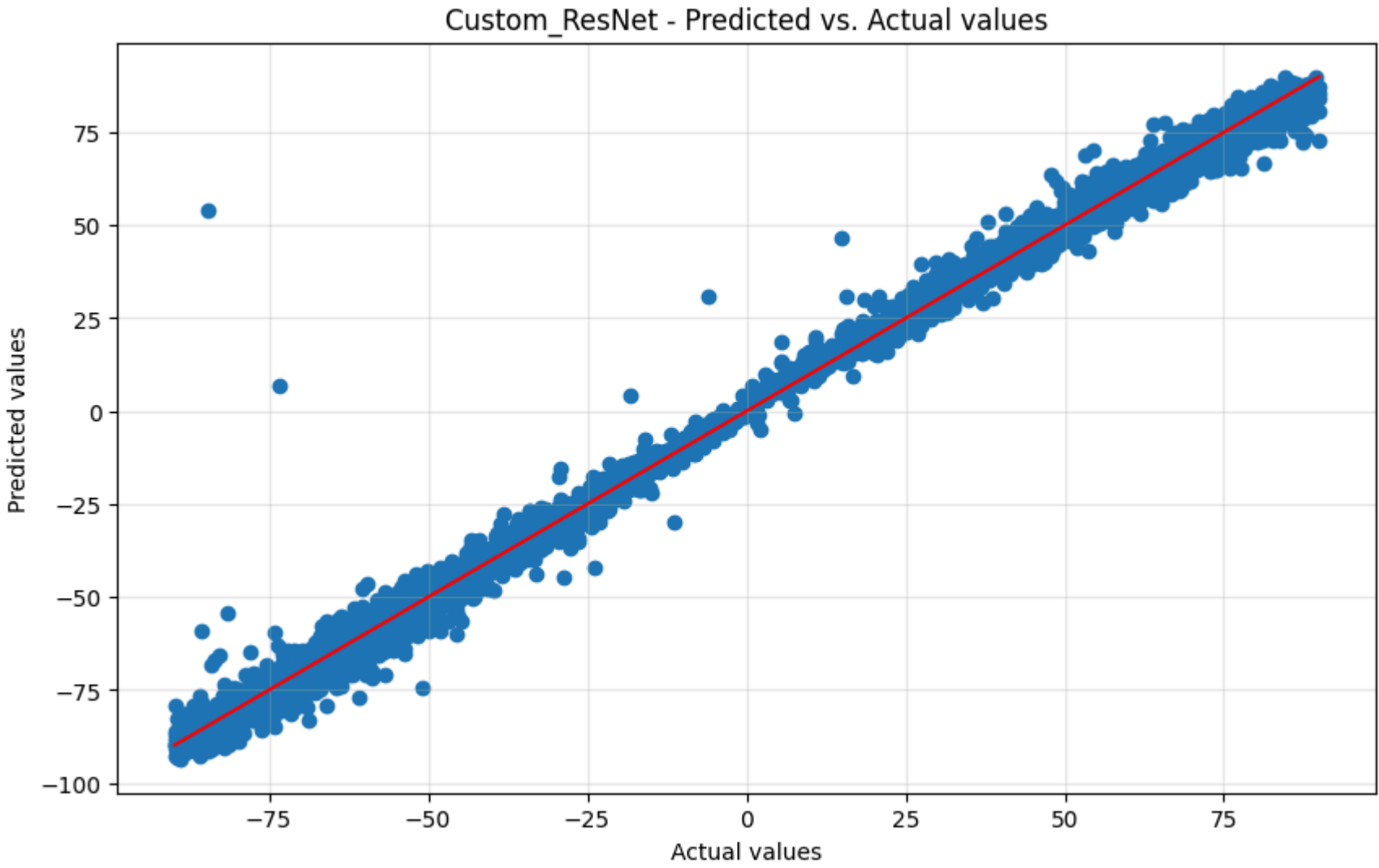

```
In [17]: # Train custom ResNet model
custom_resnet_history = custom_resnet_model.fit(
    x_training,
    y_training,
    epochs=25,
    batch_size=32,
    validation_data=(x_validation, y_validation),
    callbacks=get_callbacks('custom_resnet_yaw_regressor')
)

Epoch 1/25 98s 219ms/step - loss: 943.8820 - mae: 22.8124 - val_loss: 625.7246 - val_mae: 20.1895 - learning_rate: 0.0010
Epoch 2/25 37s 193ms/step - loss: 208.4964 - mae: 10.7408 - val_loss: 290.8302 - val_mae: 12.3517 - learning_rate: 0.0010
Epoch 3/25 37s 186ms/step - loss: 159.2943 - mae: 9.4383 - val_loss: 71.0342 - val_mae: 5.9552 - learning_rate: 0.0010
Epoch 4/25 37s 194ms/step - loss: 96.7161 - mae: 7.4336 - val_loss: 42.9124 - val_mae: 4.5573 - learning_rate: 0.0010
Epoch 5/25 37s 190ms/step - loss: 82.0193 - mae: 6.8398 - val_loss: 52.5337 - val_mae: 4.8865 - learning_rate: 0.0010
Epoch 6/25 37s 185ms/step - loss: 119.0064 - mae: 7.6501 - val_loss: 70.3172 - val_mae: 5.5226 - learning_rate: 0.0010
Epoch 7/25 37s 189ms/step - loss: 114.4217 - mae: 7.8711 - val_loss: 42.1601 - val_mae: 4.5109 - learning_rate: 0.0010
Epoch 8/25 37s 191ms/step - loss: 75.7303 - mae: 6.5749 - val_loss: 30.1953 - val_mae: 3.6526 - learning_rate: 0.0010
Epoch 9/25 37s 183ms/step - loss: 67.3867 - mae: 6.1676 - val_loss: 29.3089 - val_mae: 3.7517 - learning_rate: 0.0010
Epoch 10/25 37s 181ms/step - loss: 68.3297 - mae: 6.2555 - val_loss: 38.5016 - val_mae: 4.4753 - learning_rate: 0.0010
Epoch 11/25 37s 191ms/step - loss: 65.4184 - mae: 6.0661 - val_loss: 27.9306 - val_mae: 3.5747 - learning_rate: 0.0010
Epoch 12/25 37s 186ms/step - loss: 63.3492 - mae: 5.9891 - val_loss: 41.1242 - val_mae: 4.3339 - learning_rate: 0.0010
Epoch 13/25 37s 182ms/step - loss: 61.6350 - mae: 5.8432 - val_loss: 30.1732 - val_mae: 3.7463 - learning_rate: 0.0010
Epoch 14/25 37s 185ms/step - loss: 72.0213 - mae: 6.2140 - val_loss: 317.4700 - val_mae: 13.1315 - learning_rate: 0.0010
Epoch 15/25 37s 185ms/step - loss: 86.8366 - mae: 6.9315 - val_loss: 33.3006 - val_mae: 3.7249 - learning_rate: 0.0010
Epoch 16/25 37s 189ms/step - loss: 62.5063 - mae: 5.9191 - val_loss: 29.8077 - val_mae: 3.7961 - learning_rate: 5.0000e-04
Epoch 17/25 37s 190ms/step - loss: 57.1237 - mae: 5.6888 - val_loss: 25.3648 - val_mae: 3.4832 - learning_rate: 5.0000e-04
Epoch 18/25 37s 192ms/step - loss: 52.4690 - mae: 5.4484 - val_loss: 22.4599 - val_mae: 3.0676 - learning_rate: 5.0000e-04
Epoch 19/25 37s 194ms/step - loss: 52.3950 - mae: 5.4335 - val_loss: 23.2738 - val_mae: 3.1128 - learning_rate: 5.0000e-04
Epoch 20/25 37s 190ms/step - loss: 52.7588 - mae: 5.4078 - val_loss: 22.4012 - val_mae: 3.1377 - learning_rate: 5.0000e-04
Epoch 21/25 37s 187ms/step - loss: 51.9523 - mae: 5.3830 - val_loss: 21.1578 - val_mae: 3.0610 - learning_rate: 5.0000e-04
Epoch 22/25 37s 185ms/step - loss: 53.6316 - mae: 5.4726 - val_loss: 21.3838 - val_mae: 3.1568 - learning_rate: 5.0000e-04
Epoch 23/25 37s 184ms/step - loss: 51.3284 - mae: 5.3118 - val_loss: 21.1694 - val_mae: 3.0696 - learning_rate: 5.0000e-04
Epoch 24/25 37s 182ms/step - loss: 52.5925 - mae: 5.4163 - val_loss: 21.8546 - val_mae: 3.0607 - learning_rate: 5.0000e-04
Epoch 25/25 37s 189ms/step - loss: 50.5622 - mae: 5.3176 - val_loss: 20.8518 - val_mae: 2.8086 - learning_rate: 5.0000e-04
```

```
In [18]: # Plot training history and evaluate custom ResNet model
custom_resnet_metrics = plot_training_history(custom_resnet_history, 'Custom_ResNet')
custom_resnet_results = evaluate_model(custom_resnet_model, x_testing, y_testing, 'Custom_ResNet')
```



125/125 2s 10ms/step
==== Custom_ResNet Evaluation =====
MAE: 2.8086298986240234
MSE: 20.851770030395080
RMSE: 4.569374714190187
R-squared: 0.9929732084274292



3. Transfer Learning with ResNet50

This model utilizes a pre-trained ResNet50 model initially trained on ImageNet and fine-tuned for head yaw regression.

```
In [19]: def create_resnet50_model(input_shape=(128, 128, 3)):  
# Load pre-trained ResNet50 without top layers  
base_model = ResNet50(  
    include_top=False,  
    weights='imagenet',  
    input_shape=input_shape  
)  
  
# Freeze base model layers  
for layer in base_model.layers:  
    layer.trainable = False  
  
# Create model with custom regression head  
inputs = Input(shape=input_shape)  
x = base_model(inputs)  
x = GlobalAveragePooling2D()(x)  
x = Dense(512, activation='relu')(x)  
x = Dropout(0.5)(x)  
x = Dense(128, activation='relu')(x)  
x = Dropout(0.3)(x)  
outputs = Dense(1)(x) # Regression output for yaw angle  
  
model = Model(inputs=inputs, outputs=outputs)  
  
# Compile the model with a lower learning rate for transfer learning  
model.compile(  
    optimizer=Adam(learning_rate=0.0001),  
    loss='mse',  
    metrics=['mae']  
)  
  
return model
```

```
In [20]: # Create ResNet50 transfer learning model  
resnet50_model = create_resnet50_model()  
resnet50_model.summary()
```

Model: "functional_2"

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 128, 128, 3)	0
resnet50 (Functional)	(None, 4, 4, 2048)	23,587,712
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 2048)	0
dense_6 (Dense)	(None, 512)	1,049,088
dropout_4 (Dropout)	(None, 512)	0
dense_7 (Dense)	(None, 128)	65,664
dropout_5 (Dropout)	(None, 128)	0
dense_8 (Dense)	(None, 1)	129

Total params: 24,702,593 (94.23 MB)

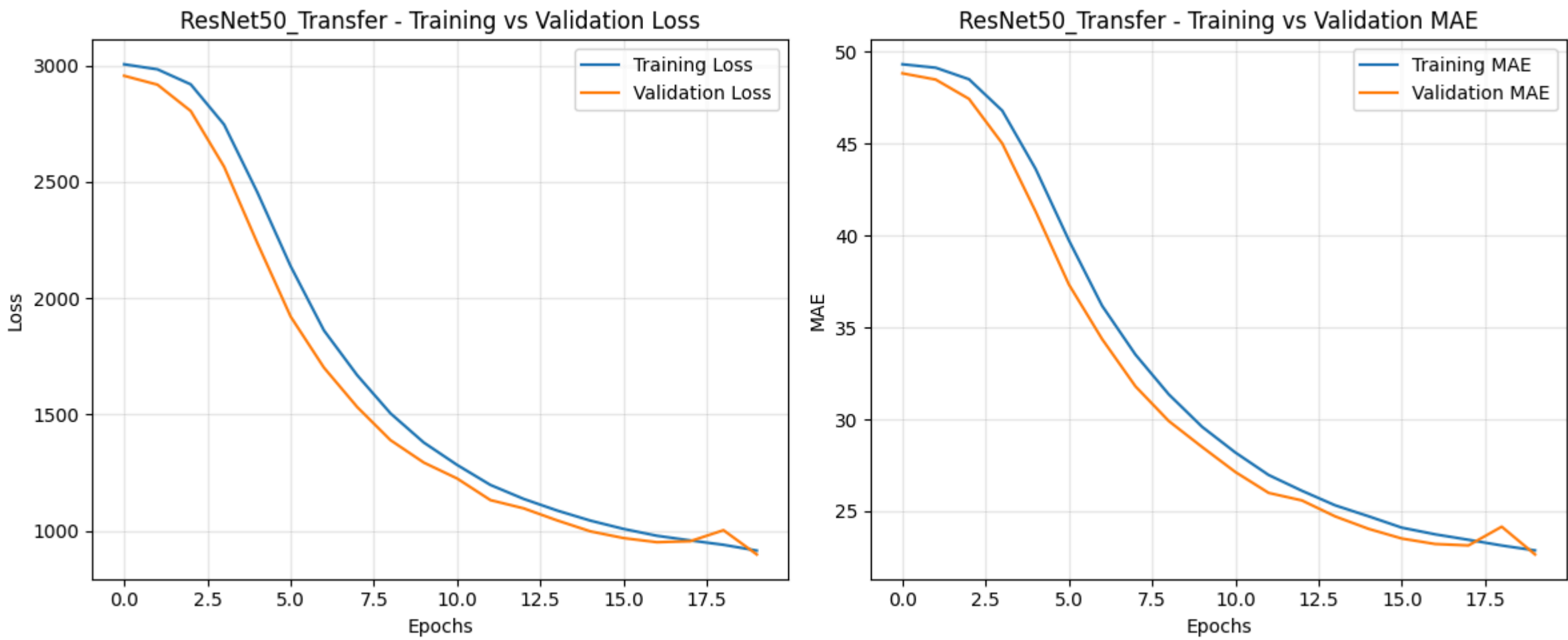
Trainable params: 1,114,881 (4.25 MB)

Non-trainable params: 23,587,712 (89.98 MB)

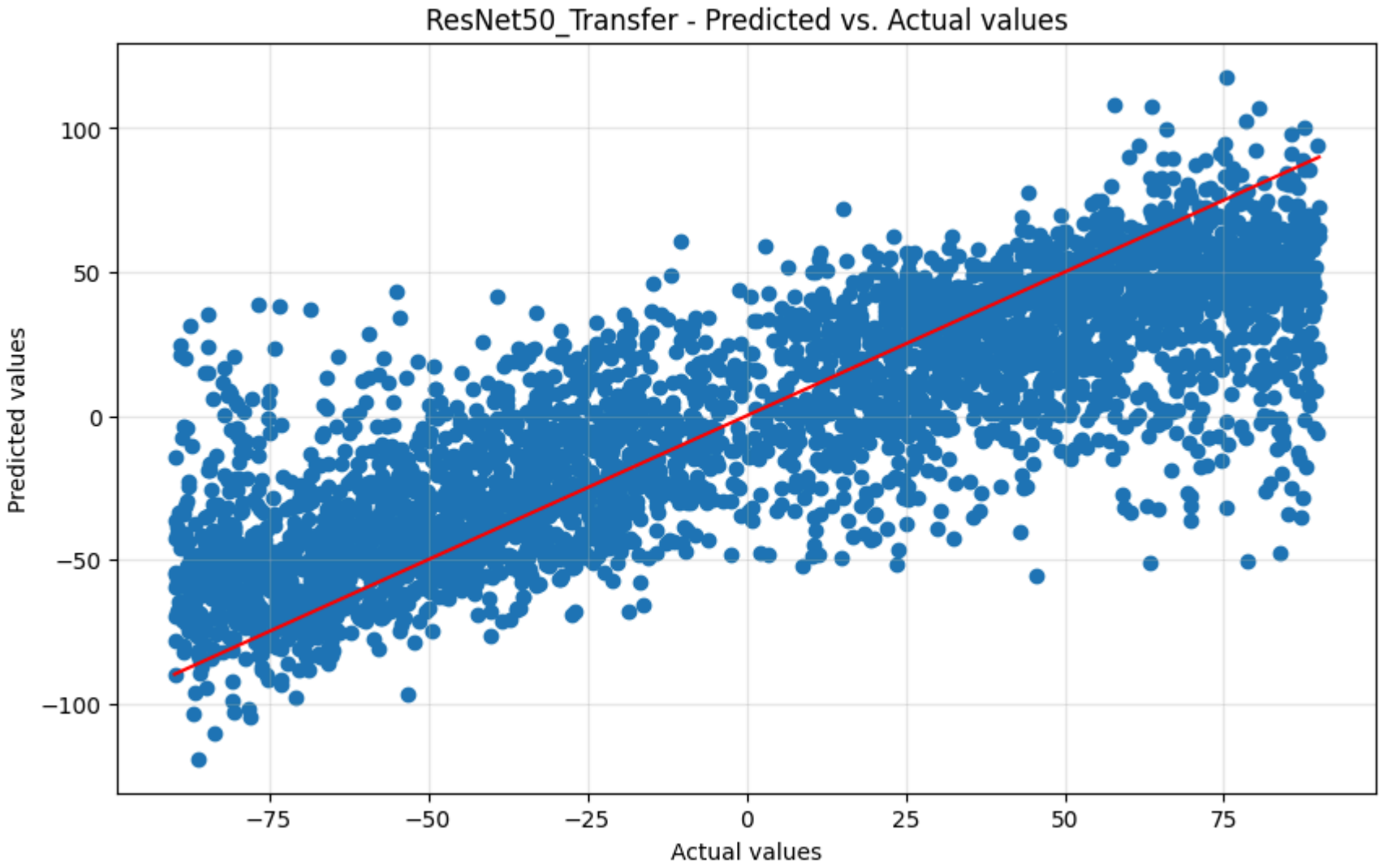
```
In [21]: # Train ResNet50 model  
resnet50_history = resnet50_model.fit(  
    x_training,  
    y_training,  
    epochs=20,  
    batch_size=32,  
    validation_data=(x_validation, y_validation),  
    callbacks=get_callbacks('resnet50_yaw_regressor')  
)  
  
Epoch 1/20 49s 105ms/step - loss: 3035.2695 - mae: 49.6388 - val_loss: 2955.3560 - val_mae: 48.8343 - learning_rate: 1.0000e-04  
Epoch 2/20 33s 87ms/step - loss: 2974.4155 - mae: 49.0206 - val_loss: 2917.2454 - val_mae: 48.4950 - learning_rate: 1.0000e-04  
Epoch 3/20 33s 88ms/step - loss: 2978.3145 - mae: 49.0618 - val_loss: 2804.0630 - val_mae: 47.4433 - learning_rate: 1.0000e-04  
Epoch 4/20 33s 87ms/step - loss: 2828.1169 - mae: 47.6481 - val_loss: 2565.1780 - val_mae: 45.0153 - learning_rate: 1.0000e-04  
Epoch 5/20 32s 86ms/step - loss: 2530.9116 - mae: 44.5296 - val_loss: 2236.7224 - val_mae: 41.3176 - learning_rate: 1.0000e-04  
Epoch 6/20 32s 85ms/step - loss: 2194.1399 - mae: 40.5039 - val_loss: 1920.8177 - val_mae: 37.3431 - learning_rate: 1.0000e-04  
Epoch 7/20 32s 86ms/step - loss: 1926.3584 - mae: 37.0646 - val_loss: 1701.3784 - val_mae: 34.3645 - learning_rate: 1.0000e-04  
Epoch 8/20 32s 84ms/step - loss: 1709.2261 - mae: 34.1342 - val_loss: 1531.9490 - val_mae: 31.7978 - learning_rate: 1.0000e-04  
Epoch 9/20 31s 84ms/step - loss: 1543.4098 - mae: 31.9949 - val_loss: 1389.5809 - val_mae: 29.9097 - learning_rate: 1.0000e-04  
Epoch 10/20 31s 83ms/step - loss: 1426.0760 - mae: 30.2144 - val_loss: 1293.3024 - val_mae: 28.4984 - learning_rate: 1.0000e-04  
Epoch 11/20 31s 83ms/step - loss: 1305.0400 - mae: 28.4620 - val_loss: 1224.9320 - val_mae: 27.1352 - learning_rate: 1.0000e-04  
Epoch 12/20 31s 84ms/step - loss: 1229.3024 - mae: 27.4532 - val_loss: 1132.0671 - val_mae: 25.9984 - learning_rate: 1.0000e-04  
Epoch 13/20 31s 84ms/step - loss: 1157.3680 - mae: 26.3735 - val_loss: 1096.5823 - val_mae: 25.5871 - learning_rate: 1.0000e-04  
Epoch 14/20 31s 84ms/step - loss: 1096.4238 - mae: 25.4519 - val_loss: 1045.0966 - val_mae: 24.7169 - learning_rate: 1.0000e-04  
Epoch 15/20 32s 85ms/step - loss: 1057.3740 - mae: 24.8879 - val_loss: 997.8649 - val_mae: 24.0422 - learning_rate: 1.0000e-04  
Epoch 16/20 31s 84ms/step - loss: 1021.0744 - mae: 24.2111 - val_loss: 968.8399 - val_mae: 23.5144 - learning_rate: 1.0000e-04  
Epoch 17/20 32s 84ms/step - loss: 975.4359 - mae: 23.5125 - val_loss: 951.1379 - val_mae: 23.2164 - learning_rate: 1.0000e-04  
Epoch 18/20 31s 83ms/step - loss: 949.3511 - mae: 23.3856 - val_loss: 955.3045 - val_mae: 23.1384 - learning_rate: 1.0000e-04  
Epoch 19/20 31s 82ms/step - loss: 935.8019 - mae: 23.1766 - val_loss: 1002.7047 - val_mae: 24.1590 - learning_rate: 1.0000e-04  
Epoch 20/20 33s 89ms/step - loss: 929.6495 - mae: 23.1523 - val_loss: 898.5840 - val_mae: 22.6488 - learning_rate: 1.0000e-04
```



```
In [22]: # Plot training history and evaluate ResNet50 model
resnet50_metrics = plot_training_history(resnet50_history, 'ResNet50_Transfer')
resnet50_results = evaluate_model(resnet50_model, x_testing, y_testing, 'ResNet50_Transfer')
```



125/125 ————— 9s 49ms/step
==== ResNet50_Transfer Evaluation =====
MAE: 22.64883884321289
MSE: 898.5838623046875
RMSE: 29.976388413294345
R-squared: 0.6971873844967651



4. Transfer Learning with MobileNetV2

This model utilizes a pre-trained MobileNetV2 model initially trained on ImageNet and fine-tuned for head yaw regression. MobileNetV2 is a lightweight architecture designed for mobile and edge devices.

```
In [23]: def create_mobilenetv2_model(input_shape=(128, 128, 3)):  
# Load pre-trained MobileNetV2 without top layers  
base_model = MobileNetV2(  
    include_top=False,  
    weights='imagenet',  
    input_shape=input_shape  
)  
  
# Freeze base model layers  
for layer in base_model.layers:  
    layer.trainable = False  
  
# Create model with custom regression head  
inputs = Input(shape=input_shape)  
x = base_model(inputs)  
x = GlobalAveragePooling2D()(x)  
x = Dense(512, activation='relu')(x)  
x = Dropout(0.5)(x)  
x = Dense(128, activation='relu')(x)  
x = Dropout(0.3)(x)  
outputs = Dense(1)(x) # Regression output for yaw angle  
  
model = Model(inputs=inputs, outputs=outputs)  
  
# Compile the model with a lower learning rate for transfer learning  
model.compile(  
    optimizer=Adam(learning_rate=0.0001),  
    loss='mse',  
    metrics=['mae']  
)  
  
return model
```

```
In [24]: # Create MobileNetV2 transfer learning model  
mobilenetv2_model = create_mobilenetv2_model()  
mobilenetv2_model.summary()
```

Model: "functional_3"

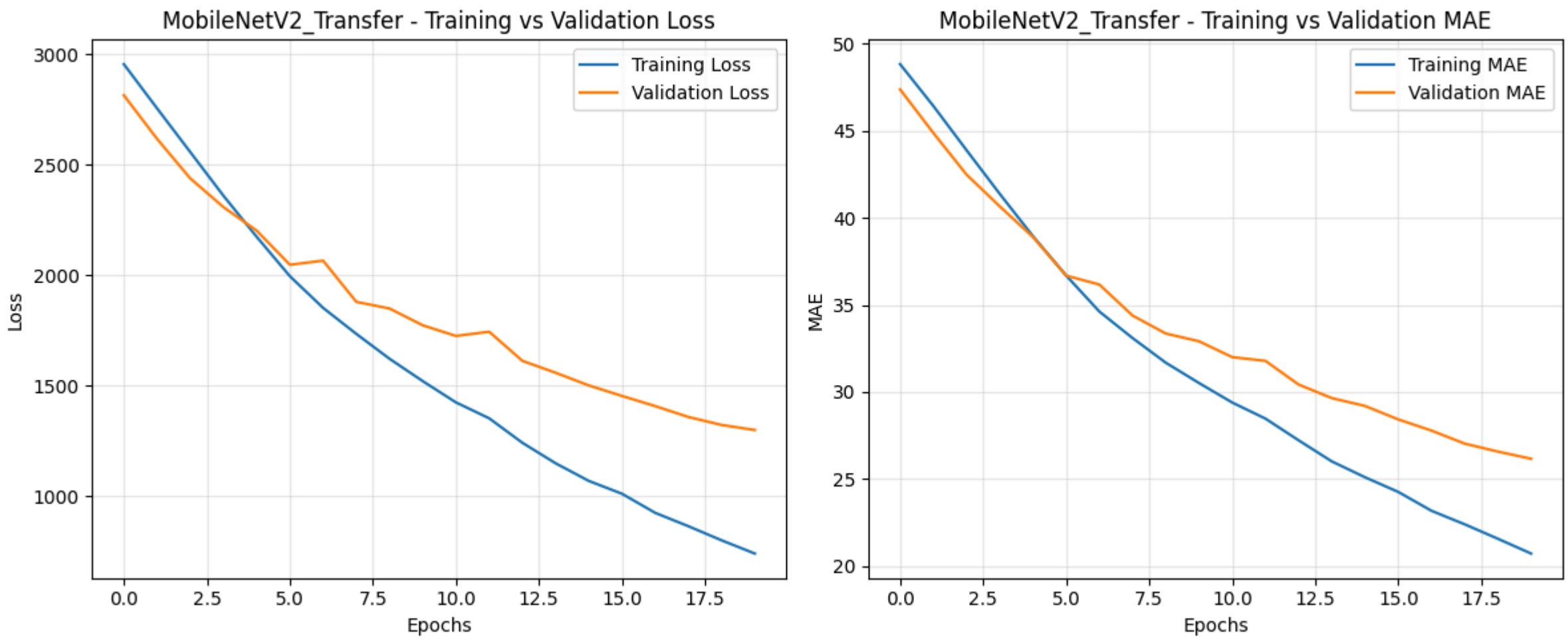
Layer (type)	Output Shape	Param #
input_layer_5 (InputLayer)	(None, 128, 128, 3)	0
mobilenetv2_1_00_128 (Functional)	(None, 4, 4, 1280)	2,257,984
global_average_pooling2d_2 (GlobalAveragePooling2D)	(None, 1280)	0
dense_9 (Dense)	(None, 512)	655,072
dropout_6 (Dropout)	(None, 512)	0
dense_10 (Dense)	(None, 128)	65,664
dropout_7 (Dropout)	(None, 128)	0
dense_11 (Dense)	(None, 1)	129

Total params: 2,979,649 (11.37 MB)
Trainable params: 721,665 (2.75 MB)
Non-trainable params: 2,257,984 (8.61 MB)

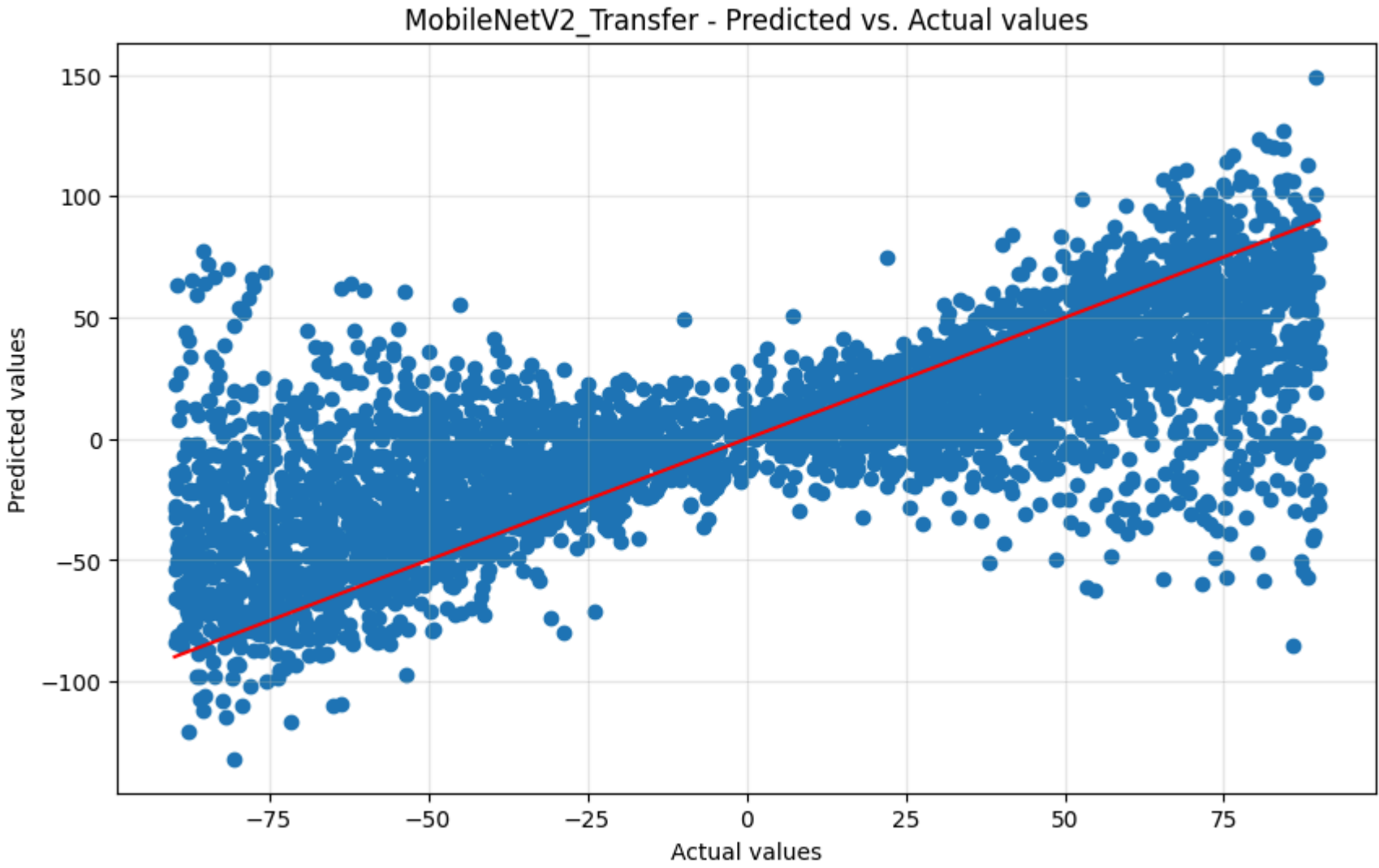
```
In [25]: # Train MobileNetV2 model  
mobilenetv2_history = mobilenetv2_model.fit(  
    x_training,  
    y_training,  
    epochs=20,  
    batch_size=32,  
    validation_data=(x_validation, y_validation),  
    callbacks=get_callbacks('mobilenetv2_yaw_regressor')  
)
```

Epoch 1/20 ————— 32s 70ms/step - loss: 3020.6262 - mae: 49.4931 - val_loss: 2813.9622 - val_mae: 47.3792 - learning_rate: 1.0000e-04
375/375 ————— 21s 55ms/step - loss: 2788.3491 - mae: 46.8741 - val_loss: 2616.9583 - val_mae: 44.8822 - learning_rate: 1.0000e-04
Epoch 2/20 ————— 21s 55ms/step - loss: 2599.6465 - mae: 44.5030 - val_loss: 2438.0796 - val_mae: 42.4066 - learning_rate: 1.0000e-04
375/375 ————— 21s 55ms/step - loss: 2384.5894 - mae: 41.8362 - val_loss: 2308.2466 - val_mae: 40.6625 - learning_rate: 1.0000e-04
Epoch 3/20 ————— 21s 55ms/step - loss: 2188.6396 - mae: 39.4071 - val_loss: 2201.7529 - val_mae: 38.8998 - learning_rate: 1.0000e-04
375/375 ————— 21s 55ms/step - loss: 2010.0505 - mae: 37.0164 - val_loss: 2047.3827 - val_mae: 36.6838 - learning_rate: 1.0000e-04
Epoch 4/20 ————— 20s 52ms/step - loss: 1907.5826 - mae: 35.2605 - val_loss: 2066.2319 - val_mae: 36.1702 - learning_rate: 1.0000e-04
375/375 ————— 20s 52ms/step - loss: 1768.8352 - mae: 33.3970 - val_loss: 1879.7620 - val_mae: 34.3882 - learning_rate: 1.0000e-04
Epoch 5/20 ————— 19s 51ms/step - loss: 1675.4338 - mae: 32.3205 - val_loss: 1849.6575 - val_mae: 33.3557 - learning_rate: 1.0000e-04
375/375 ————— 20s 52ms/step - loss: 1500.9559 - mae: 30.3540 - val_loss: 1773.7985 - val_mae: 32.9119 - learning_rate: 1.0000e-04
Epoch 6/20 ————— 21s 55ms/step - loss: 1398.4276 - mae: 29.1160 - val_loss: 1725.6754 - val_mae: 31.9998 - learning_rate: 1.0000e-04
375/375 ————— 19s 52ms/step - loss: 1347.5316 - mae: 28.5241 - val_loss: 1744.7717 - val_mae: 31.7869 - learning_rate: 1.0000e-04
Epoch 7/20 ————— 20s 54ms/step - loss: 1226.5918 - mae: 27.1883 - val_loss: 1613.0826 - val_mae: 30.4241 - learning_rate: 1.0000e-04
375/375 ————— 20s 53ms/step - loss: 1142.8486 - mae: 25.9514 - val_loss: 1559.2855 - val_mae: 29.6462 - learning_rate: 1.0000e-04
Epoch 8/20 ————— 19s 52ms/step - loss: 1070.1232 - mae: 25.1599 - val_loss: 1501.8806 - val_mae: 29.1952 - learning_rate: 1.0000e-04
375/375 ————— 21s 55ms/step - loss: 1028.8353 - mae: 24.3984 - val_loss: 1454.2527 - val_mae: 28.4227 - learning_rate: 1.0000e-04
Epoch 9/20 ————— 21s 55ms/step - loss: 918.1601 - mae: 22.9610 - val_loss: 1408.3715 - val_mae: 27.7809 - learning_rate: 1.0000e-04
375/375 ————— 21s 55ms/step - loss: 875.5634 - mae: 22.5004 - val_loss: 1359.1010 - val_mae: 27.0316 - learning_rate: 1.0000e-04
Epoch 10/20 ————— 21s 55ms/step - loss: 794.7761 - mae: 21.4717 - val_loss: 1323.0389 - val_mae: 26.5746 - learning_rate: 1.0000e-04
375/375 ————— 21s 56ms/step - loss: 746.4357 - mae: 20.7780 - val_loss: 1300.3105 - val_mae: 26.1620 - learning_rate: 1.0000e-04


```
In [26]: # Plot training history and evaluate MobileNetV2 model
mobilenetv2_metrics = plot_training_history(mobilenetv2_history, 'MobileNetV2_Transfer')
mobilenetv2_results = evaluate_model(mobilenetv2_model1, x_testing, y_testing, 'MobileNetV2_Transfer')
```



```
125/125 ----- 5s 22ms/step
===== MobileNetV2_Transfer Evaluation =====
MAE: 26.162017822265625
MSE: 1300.3101806640025
RMSE: 36.059813929970055
R-squared: 0.5618100166320001
```



Compare Model Performance

```
In [27]: # Create a DataFrame to compare metrics
all_results = pd.DataFrame([
    attention_results,
    custom_resnet_results,
    resnet50_results,
    mobilenetv2_results
])

print("===== Model Performance Comparison =====")
all_results.set_index('model')
```

Out[27]:

	mae	mse	rmse	r2
model				
Attention_CNN	2.585053	12.270391	3.502912	0.995865
Custom_ResNet	2.800630	20.851778	4.566375	0.992973
ResNet50_Transfer	22.648838	898.583862	29.976388	0.697187
MobileNetV2_Transfer	26.162018	1300.310161	36.059814	0.561810

```
In [28]: # Visualize comparison with bar plots
plt.figure(figsize=(14, 10))

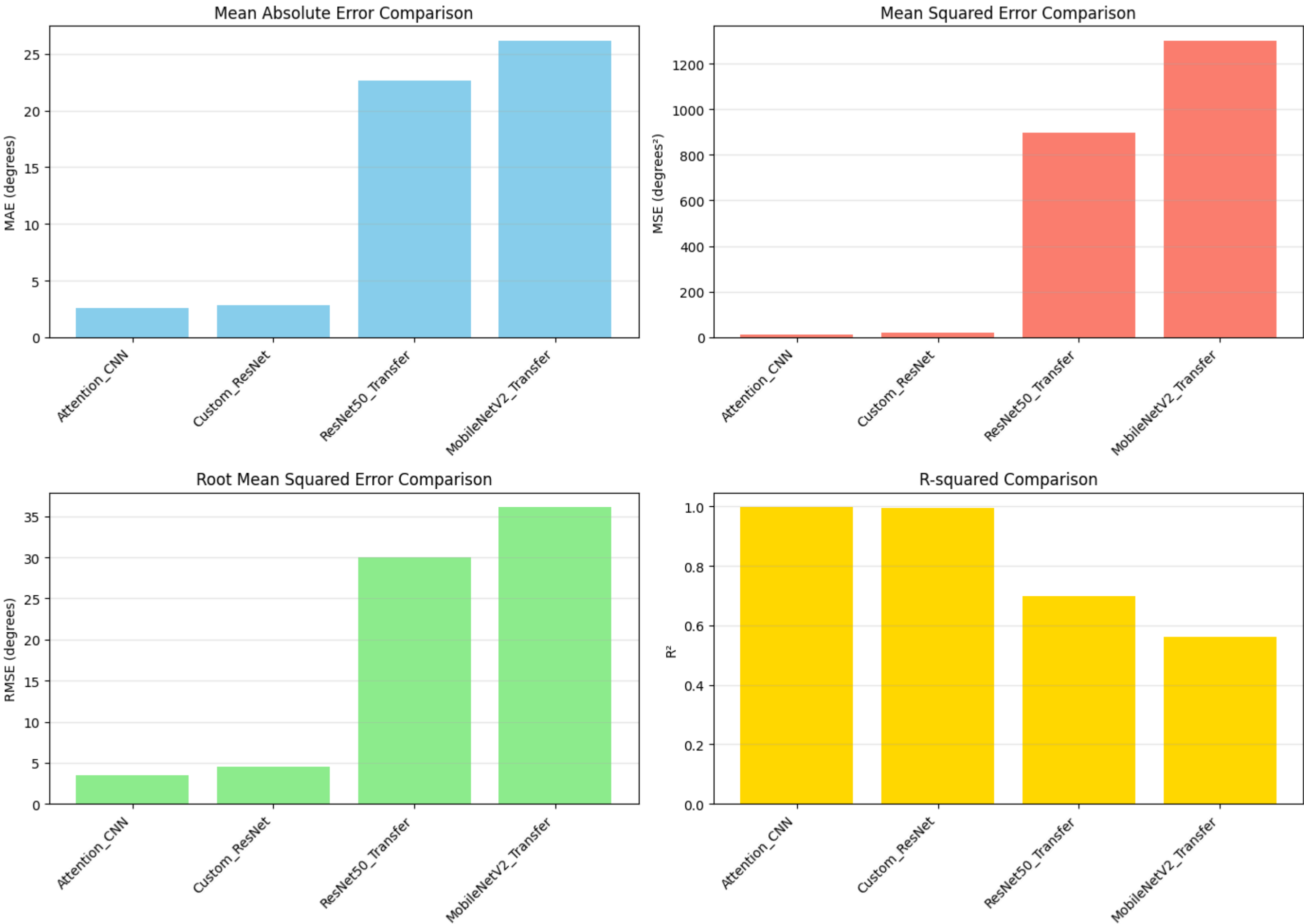
# MAE comparison
plt.subplot(2, 2, 1)
plt.bar(all_results['model'], all_results['mae'], color='skyblue')
plt.title('Mean Absolute Error Comparison')
plt.ylabel('MAE (degrees)')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3)

# MSE comparison
plt.subplot(2, 2, 2)
plt.bar(all_results['model'], all_results['mse'], color='salmon')
plt.title('Mean Squared Error Comparison')
plt.ylabel('MSE (degrees^2)')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3)

# RMSE comparison
plt.subplot(2, 2, 3)
plt.bar(all_results['model'], all_results['rmse'], color='lightgreen')
plt.title('Root Mean Squared Error Comparison')
plt.ylabel('RMSE (degrees)')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3)

# R^2 comparison
plt.subplot(2, 2, 4)
plt.bar(all_results['model'], all_results['r2'], color='gold')
plt.title('R-squared Comparison')
plt.ylabel('R^2')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig('../model/experiments/model_comparison.png')
plt.show()
```



Conclusion

In this notebook, we evaluated four different CNN architectures for head yaw angle regression:

- Attention-based CNN with CBAM** - This model uses attention mechanisms to focus on the most relevant parts of the face for yaw angle prediction.
- Custom ResNet-based CNN** - A custom architecture with residual connections that help with gradient flow in deep networks.
- ResNet50 (Transfer Learning)** - Uses a pre-trained ResNet50 model and fine-tunes it for our specific regression task.
- MobileNetV2 (Transfer Learning)** - Uses a pre-trained MobileNetV2 model, offering a more lightweight architecture suitable for deployment on edge devices.

Based on the evaluation metrics, we can determine which model performs best for the head yaw regression task. The comparison considers accuracy (MAE, MSE, RMSE) and model efficiency/complexity.